

Supplementary Technical Report: Proxy-Guided Sampling for Approximate Graph Aggregation with Machine Learning Predicates

Anonymous Authors (Submission ID: 975)

This technical report serves as a supplementary document to our main submission. Due to strict space limitations in the conference format, we provide here:

1. A detailed real-world business motivation and case study (Section 1);
2. The end-to-end cost model and per-operation latency breakdown (Section 2);
3. The rigorous mathematical proof of uniform tree-homomorphism sampling (Section 3);
4. The complete proof of consistency and asymptotic bias for the AVG estimator under the joint asymptotic regime (Section 4).

1 Real-World Motivation and Business Case Study

This section details a concrete, high-value business scenario demonstrating that Graph Aggregation with ML predicates (GA-ML) is an urgent requirement in modern AI-augmented databases, rather than an artificial extension of standard graph queries.

The Real-World Need for Multi-Modal ML on Graphs. Modern enterprises (e.g., Amazon, Alibaba) manage their data as heterogeneous property graphs (comprising User, Product, and Review vertices). However, crucial business insights are often locked inside unstructured attributes (images and text) attached to these vertices. For instance, a quality-control analyst investigating user dissatisfaction needs to query: *"Find all transactions where a user purchased a wooden furniture product and left an angry/negative review."*

Traditional relational filters cannot express this. It strictly requires multi-modal ML predicates: a Computer Vision (CV) model to verify "wooden furniture" from product images, and an NLP sentiment model to detect "angry" emotions in review texts.

The Business Value of Graph Aggregation (COUNT/SUM/AVG). In enterprise scenarios, merely retrieving a massive list of matched subgraphs is unhelpful for decision-making. Analysts fundamentally need macro-level aggregations over these topological structures. As detailed in our Introduction, applying an aggregation function λ provides distinct business KPIs:

- $\lambda = \text{COUNT}$: Quantifies the total transaction volume of these controversial orders.
- $\lambda = \text{SUM}$: Calculates the total sales revenue at risk (by aggregating the price attribute of the matched subgraphs).
- $\lambda = \text{AVG}$: Identifies the average unit price of the criticized items, helping to pinpoint if the issue lies in high-end or low-end markets.

2 End-to-End Cost Model and Latency Analysis

The total execution cost of a GA-ML query in PROXY can be decoupled into offline setup cost, which can be amortized across a workload, and online query cost.

2.1 Offline Setup Cost (Amortized Across Queries)

The offline cost comprises model initialization and optional proxy fine-tuning:

- **Zero-Shot Plug-and-Play:** PROXY supports off-the-shelf pre-trained models from Hugging Face (e.g., Oracle model: `microsoft/deberta-v2-xxlarge-mnli`, Proxy model: `sileod/deberta-v3-base`). For these models, C_{offline} consists primarily of one-time model and runtime initialization/loading.
- **Task-Specific Fine-Tuning (Optional):** To systematically evaluate the effect of proxy degradation (e.g., $F_1 \in [0.34, 0.89]$), we lightly fine-tuned base backbones (e.g., `microsoft/deberta-v3-base`) using a small, disjoint background split (500–1000 instances). This offline process takes only a few minutes. Across a workload of N_q queries, the amortized contribution is C_{offline}/N_q , which becomes negligible as N_q grows.

2.2 Online Query Cost and Per-Operation Latency

The online latency C_{online} consists of graph structural preparation, proxy scoring, and expensive Oracle verification:

$$C_{\text{online}} = \underbrace{C_{\text{CS}} + Kc_{\text{tree}}}_{C_{\text{prep}}: \text{In-Memory Graph Operations}} + \underbrace{|\hat{\Psi}| \bar{c}_{\text{proxy}} + m \bar{c}_{\text{eff}}}_{\text{ML Inference}}. \quad (1)$$

Here, C_{CS} is the one-time Candidate Space construction cost for the query, K is the number of tree-homomorphism samples used for structural weight estimation, c_{tree} is the per-sample tree-sampling cost, and $\bar{c}_{\text{eff}}$ is the expected Oracle cost per sampled projection after accounting for short-circuiting and predicate-level cache reuse.

To quantify the relative cost of the different operations, we benchmarked a typical Natural Language Inference (NLI) configuration (Proxy: `deberta-v3-base` vs. Oracle: `deberta-v2-xxlarge`) on a single NVIDIA RTX 3090 GPU:

1. C_{prep} : Constructing the Candidate Space (CS) summary and drawing $K = 60,000$ tree-homomorphism samples takes approximately 1000 ms in total in our benchmark. Equivalently, the measured end-to-end preparation overhead is about $16.7 \mu\text{s}$ per sampled tree homomorphism when the CS construction cost is amortized over the K samples. We use the measured aggregate preparation cost $C_{\text{prep}} \approx 1000$ ms in the break-even analysis below rather than interpreting $16.7 \mu\text{s}$ as the isolated tree-sampling latency.
2. $|\hat{\Psi}| \bar{c}_{\text{proxy}}$: The lightweight proxy model is evaluated on the distinct sampled projections. With batched GPU inference, the observed per-item latency is approximately 1.0–1.8 ms depending on batch size, corresponding to roughly 550–960 items/s.
3. $m \bar{c}_{\text{eff}}$: The heavy Oracle dominates the ML execution cost. The 1.5B-parameter Oracle processes approximately 13 items/s in the benchmark, corresponding to about 78 ms per item. Under our typical sampling regime, Oracle verification accounts for more than 95% of the online latency.

Because Oracle inference is substantially more expensive than proxy inference (e.g., approximately 78 ms vs. 1.8 ms per item in the benchmark), limiting the number of expensive Oracle evaluations is essential for reducing total query latency under a fixed Oracle budget.

2.3 Theoretical Break-Even Analysis: PROXY vs. Exact Evaluation

A key practical question is: *Under what workload size and Oracle budget does PROXY become faster than direct exact evaluation?*

To make the comparison conservative, we do not compare PROXY against naive exhaustive execution. Instead, we define an **Optimized Exact Engine** that eliminates redundant structural-semantic evaluations whenever the same semantic predicate input is encountered.

Let

$$N = |\Psi|$$

denote the number of distinct semantic projections in the exact result space. Its cost can be written as

$$C_E = C_{\text{match}} + N \bar{c}_E, \quad (2)$$

where C_{match} is the cost of exhaustive structural matching and $\bar{c}_E$ is the average effective Oracle cost per distinct projection under the exact execution strategy, including any short-circuiting and predicate-level caching.

In contrast, PROXY avoids exhaustive structural matching by constructing the Candidate Space and drawing K tree-homomorphism samples. Let $\hat{\Psi} \subseteq \Psi$ be the set of distinct semantic projections encountered by the sampler and let

$$\hat{N} = |\hat{\Psi}|.$$

For the break-even analysis, we use the simplified parameterization

$$m = \alpha \hat{N}, \quad \alpha \in (0, 1),$$

where α denotes the fraction of sampled projections that receive Oracle verification. Then

$$C_P = C_{\text{prep}} + \hat{N} \bar{c}_P + \alpha \hat{N} \bar{c}_O^{(q)}, \quad (3)$$

where $\bar{c}_P$ is the average proxy inference cost per projection and $\bar{c}_O^{(q)}$ is the expected Oracle cost per sampled projection under the proposal distribution q .

Therefore,

$$\Delta = C_E - C_P = (C_{\text{match}} - C_{\text{prep}}) + N \bar{c}_E - \hat{N} \bar{c}_P - \alpha \hat{N} \bar{c}_O^{(q)}. \quad (4)$$

A positive Δ indicates that PROXY is faster than the exact baseline.

2.3.1 Case 1: Single-Predicate Break-Even Dynamics

For a single primitive predicate, there is no early short-circuiting. Moreover, when Ψ is defined on the predicate's inference vertices, one Oracle evaluation is required per distinct projection under the optimized exact engine. Hence,

$$\bar{c}_E = \bar{c}_O^{(q)} = c_{\text{oracle}}, \quad \bar{c}_P = c_{\text{proxy}}.$$

To obtain a conservative lower bound for PROXY, we make two assumptions that favor the exact baseline:

1. $C_{\text{match}} = 0$, i.e., we ignore the positive cost of exhaustive structural matching;
2. $N = \hat{N}$, i.e., we assume that PROXY must process the same number of distinct projections as exact evaluation.

Under these assumptions, Eq. (4) becomes

$$Nc_{\text{oracle}} > Nc_{\text{proxy}} + \alpha Nc_{\text{oracle}}. \quad (5)$$

Equivalently,

$$c_{\text{proxy}} < (1 - \alpha)c_{\text{oracle}}. \quad (6)$$

Defining the Oracle-to-Proxy speedup as

$$S_o = \frac{c_{\text{oracle}}}{c_{\text{proxy}}},$$

we obtain the minimum speedup requirement:

$$S_o > \frac{1}{1 - \alpha}. \quad (7)$$

Speedup Margin. At $\alpha = 10\%$,

$$S_o > \frac{1}{0.9} \approx 1.11 \times .$$

In our benchmark, $c_{\text{oracle}} \approx 78$ ms and $c_{\text{proxy}} \approx 1.8$ ms, giving

$$S_o \approx 43.3 \times ,$$

which is substantially larger than the minimum $1.11 \times$ requirement. Thus, under this conservative equal-cardinality setting, the proxy inference overhead is small relative to the Oracle cost that it replaces.

Critical Break-Even Projection Cardinality. We next retain the fixed preparation cost while making the most conservative assumption $C_{\text{match}} = 0$. Under the equal-cardinality setting $N = \hat{N}$, PROXY becomes faster when

$$C_{\text{prep}} < N((1 - \alpha)c_{\text{oracle}} - c_{\text{proxy}}).$$

Therefore, the critical projection cardinality is

$$N^* = \frac{C_{\text{prep}}}{(1 - \alpha)c_{\text{oracle}} - c_{\text{proxy}}}. \quad (8)$$

Using the measured aggregate preparation cost

$$C_{\text{prep}} \approx 1000 \text{ ms},$$

together with $\alpha = 0.1$, $c_{\text{oracle}} = 78$ ms, and $c_{\text{proxy}} = 1.8$ ms,

$$N^* = \frac{1000}{0.9(78) - 1.8} = \frac{1000}{68.4} \approx 14.6.$$

Thus,

$$\boxed{N^* \approx 15}.$$

Interpretation. This value should be understood as an illustrative hardware-grounded break-even point under the intentionally conservative assumptions above, rather than as a universal threshold. It shows that, when Oracle inference is tens of milliseconds per item, the fixed structural preparation cost can be amortized after only a small number of distinct projections. For workloads with hundreds or thousands of projections, the preparation overhead becomes relatively small compared with the Oracle computation.

2.3.2 Case 2: Multi-Predicate Break-Even Dynamics

For a conjunctive predicate

$$\mathcal{F} = \bigwedge_{i=1}^k \mathcal{F}_i,$$

the effective Oracle cost depends on predicate ordering, short-circuiting, proposal-induced distribution shift, and predicate-level cache reuse.

Let $\bar{c}_E$ denote the measured or analytically estimated average cost of evaluating one distinct projection under the exact execution strategy. This quantity already incorporates the exact engine's predicate ordering, short-circuit behavior, and caching.

Similarly, let

$$\bar{c}_O^{(q)}$$

denote the expected Oracle verification cost of one sampled projection under PROXY's proposal distribution q . We can write

$$\bar{c}_O^{(q)} = \sum_{i=1}^k \rho_{i-1}^{(q)} c_i, \quad \rho_0^{(q)} = 1, \quad (9)$$

where $\rho_{i-1}^{(q)}$ is the probability, under q , that the first $i-1$ predicates pass and the i -th Oracle is therefore reached. Importantly, $\rho_{i-1}^{(q)}$ is defined using the *actual Oracle outcomes*, rather than proxy predictions, because the Oracle execution path is determined by the true predicate results.

Likewise, one may define

$$\bar{c}_E = \sum_{i=1}^k \rho_{i-1}^{(E)} c_i$$

for the corresponding exact-execution reach probabilities $\rho_{i-1}^{(E)}$.

Under the simplified $m = \alpha \hat{N}$ parameterization, Eq. (4) becomes

$$\Delta = (C_{\text{match}} - C_{\text{prep}}) + N \bar{c}_E - \hat{N} \sum_{i=1}^k c_p^i - \alpha \hat{N} \sum_{i=1}^k \rho_{i-1}^{(q)} c_i. \quad (10)$$

This expression makes clear that the multi-predicate break-even boundary is workload-dependent. In particular, it depends on:

- **Predicate ordering and selectivity.** Early short-circuiting can substantially reduce the number of expensive later-predicate evaluations. However, importance sampling may shift the sampled distribution toward positive projections, so the Oracle reach probabilities under q need not equal those under the natural exact distribution.
- **Predicate dependence.** Correlated predicates can cause the proposal distribution and actual Oracle reach probabilities to differ substantially from those predicted by a product of marginal proxy probabilities. This affects efficiency, but not the unbiasedness of the final inverse-probability-weighted estimator.
- **Predicate-level input overlap.** Distinct semantic projections may share the same input to a particular primitive predicate. Cache reuse can therefore reduce the realized cost of both exact and sampled execution. The magnitude of this benefit depends on the distribution of repeated predicate inputs rather than solely on the number of full projections.

Budget Parameterization. For a simplified analytical break-even illustration, we parameterize the expensive Oracle budget as

$$m = \alpha \hat{N}.$$

The actual PROXY implementation does not allocate Oracle samples uniformly across projections. Instead, it determines per-stratum allocations m_i using the surrogate-based allocation strategy described in Section V-E of the main paper. The simplified α -parameterization is used only to make the break-even boundary interpretable.

2.4 Operational Boundary and Amortization Scope

The above model identifies the regimes in which PROXY is expected to be advantageous.

Favorable Regimes. PROXY is particularly attractive when:

1. the graph contains substantial structural overlap, so that the number of structural embeddings $|\Phi(Q, G)|$ is much larger than the number of semantically relevant projections $|\Psi|$;
2. Oracle inference is substantially more expensive than proxy inference;
3. the workload operates under a constrained Oracle budget, so exhaustive semantic verification is costly.

Unfavorable Regimes. Direct exact evaluation can become preferable when:

1. the projection space is very small relative to the fixed preparation overhead;
2. Oracle predicates are computationally cheap, making proxy scoring and structural preparation difficult to amortize;
3. accuracy requirements force the Oracle sample fraction toward exhaustive evaluation, i.e., $\alpha \rightarrow 1$;
4. the workload is highly dynamic or consists primarily of one-off queries, so the optional offline proxy-training cost and reusable graph preparation cannot be effectively amortized.

Overall, the break-even analysis is intended to characterize the *operational boundary* of PROXY, not to claim a universal projection-cardinality threshold. The dominant determinants are the relative costs of structural preparation, proxy inference, and expensive Oracle verification, together with the degree of semantic projection reduction and the actual short-circuit/cache behavior of the workload.

3 Proof of Uniform Tree-Homomorphism Sampling

In Section IV-C of the main paper, we proposed a top-down sampling procedure to draw a tree homomorphism $h \in \Omega$ over the Candidate Space (CS) summary. Below, we formally prove that this procedure ensures every valid homomorphism in Ω is sampled uniformly at random with exact probability $1/|\Omega|$.

Sampling Procedure Recap. Let T be a spanning tree of the query graph Q with root r . The number of homomorphisms of the subtree of T rooted at u , given u is mapped to v , is recursively calculated via dynamic programming:

$$H(u, v) = \begin{cases} 1 & \text{if } u \text{ is a leaf in } T \\ \prod_{u' \in \text{ch}(u)} \sum_{v' \in C(u' | u \rightarrow v)} H(u', v') & \text{if } u \text{ is an internal node} \end{cases} \quad (11)$$

The total number of homomorphisms is $|\Omega| = \sum_{v \in C(r)} H(r, v)$.

Our sampling procedure draws $h(r) = v_r \in C(r)$ with probability $P(r \rightarrow v_r) = \frac{H(r, v_r)}{|\Omega|}$. Then, iteratively for each mapped query vertex $u \rightarrow v$ and each of its unmapped children $u' \in \text{ch}(u)$, it selects a candidate data vertex $v' \in C(u' | u \rightarrow v)$ with probability proportional to $H(u', v')$:

$$P(u' \rightarrow v' | u \rightarrow v) = \frac{H(u', v')}{\sum_{v'' \in C(u' | u \rightarrow v)} H(u', v'')} \quad (12)$$

Theorem 1 (Uniform Tree Sampling). *For any complete valid tree homomorphism $h \in \Omega$, the probability of generating h using the proposed top-down sampling procedure is exactly $P(h) = \frac{1}{|\Omega|}$.*

Proof. Let T_u denote the subtree rooted at node u . We first prove by structural induction (bottom-up) that the conditional probability of sampling the exact partial homomorphism $h|_{T_u}$ for the subtree T_u , given that its root u is already mapped to $h(u)$, is exactly $\frac{1}{H(u, h(u))}$. Let this conditional probability be denoted as $P(T_u \text{ matches } h | u \rightarrow h(u))$.

Base Case: If u is a leaf in T , there are no children to map. The subtree T_u consists only of u . Given $u \rightarrow h(u)$, the mapping is trivially complete with probability 1. By definition, $H(u, h(u)) = 1$. Thus, $P(T_u \text{ matches } h | u \rightarrow h(u)) = 1 = \frac{1}{1} = \frac{1}{H(u, h(u))}$. The base case holds.

Inductive Step: Assume u is an internal node, and the property holds for all of its children $u' \in \text{ch}(u)$. The probability of generating the exact subtree mapping $h|_{T_u}$ given $u \rightarrow h(u)$ is the product of the probabilities of independently mapping each child $u' \rightarrow h(u')$ and recursively matching their respective subtrees $T_{u'}$.

$$P(T_u \text{ matches } h | u \rightarrow h(u)) = \prod_{u' \in \text{ch}(u)} \left[P(u' \rightarrow h(u') | u \rightarrow h(u)) \times P(T_{u'} \text{ matches } h | u' \rightarrow h(u')) \right]$$

By substituting the sampling probability $P(u' \rightarrow h(u') | u \rightarrow h(u))$ and applying the inductive hypothesis $P(T_{u'} \text{ matches } h | u' \rightarrow h(u')) = \frac{1}{H(u', h(u'))}$, we obtain:

$$\begin{aligned} P(T_u \text{ matches } h | u \rightarrow h(u)) &= \prod_{u' \in \text{ch}(u)} \left[\frac{H(u', h(u'))}{\sum_{v'' \in C(u' | u \rightarrow h(u))} H(u', v'')} \times \frac{1}{H(u', h(u'))} \right] \\ &= \prod_{u' \in \text{ch}(u)} \frac{1}{\sum_{v'' \in C(u' | u \rightarrow h(u))} H(u', v'')} \\ &= \frac{1}{\prod_{u' \in \text{ch}(u)} \sum_{v'' \in C(u' | u \rightarrow h(u))} H(u', v'')} \end{aligned}$$

By the recursive definition of $H(u, v)$ from Eq. 2, the denominator is precisely $H(u, h(u))$. Therefore:

$$P(T_u \text{ matches } h | u \rightarrow h(u)) = \frac{1}{H(u, h(u))}$$

This completes the induction.

Final Probability Calculation: The entire homomorphism h is formed by first mapping the root r to $h(r)$ and then recursively mapping the subtree T_r . The total probability $P(h)$ is:

$$\begin{aligned} P(h) &= P(r \rightarrow h(r)) \times P(T_r \text{ matches } h \mid r \rightarrow h(r)) \\ &= \frac{H(r, h(r))}{|\Omega|} \times \frac{1}{H(r, h(r))} \\ &= \frac{1}{|\Omega|} \end{aligned}$$

Thus, every valid homomorphism $h \in \Omega$ is sampled with equal probability $\frac{1}{|\Omega|}$. This rigorous bottom-up structural induction completes the proof of uniform sampling. $\square$

4 Proof of Consistency and Asymptotic Bias of the AVG Estimator

In Section VI of the main paper, we proposed the AVG ratio estimator $\hat{\tau}_{\text{avg}} = \hat{\tau}_{\text{sum}} / \hat{\tau}_{\text{count}}$. Here, we provide the complete, mathematically rigorous proof of its consistency and asymptotic bias under the joint asymptotic regime $(K, m \rightarrow \infty)$. *Note: Reference numbers to preceding theorems (e.g., Theorem 4, Theorem 5) refer to those in the main manuscript.*

Theorem 2 (Consistency and asymptotic bias of AVG). *Let $\tau_{\text{count}} \geq c_0 > 0$ and Ψ be the fixed, finite semantic projection space. For $x \in \{\text{sum}, \text{count}\}$, assume step-1 weights satisfy $|\hat{w}_x(\psi)| \leq W$ with unbiased estimators yielding variance $\mathcal{O}(K^{-1})$. Let step-2 intra-stratum probabilities be $q_i(\psi) \geq \varepsilon > 0$ with sample sizes $m_i = \beta_i m$ ($\beta_i > 0$). To ensure global boundedness, we introduce a fallback value 0 when the denominator is pathologically small: $\hat{\tau}_{\text{avg}} = \hat{\tau}_{\text{sum}} / \hat{\tau}_{\text{count}}$ if $\hat{\tau}_{\text{count}} \geq c_0/2$, and 0 otherwise.*

Under the joint asymptotic regime $K, m \rightarrow \infty$, $\hat{\tau}_{\text{avg}} \xrightarrow{p} \tau_{\text{avg}}$, and the asymptotic bias satisfies $\mathbb{E}[\hat{\tau}_{\text{avg}}] - \tau_{\text{avg}} = \mathcal{O}(m^{-1}) + \mathcal{O}(K^{-1})$. Consequently, with the regularity condition $m = o(K)$, the weight-estimation error vanishes asymptotically faster, yielding:

$$\mathbb{E}[\hat{\tau}_{\text{avg}}] - \tau_{\text{avg}} = \mathcal{O}(m^{-1}). \quad (13)$$

Proof. For $x \in \{\text{sum}, \text{count}\}$, we formalize step-1 weights by defining $\tilde{w}_x(\psi) = \hat{w}_x(\psi)$ if sampled, and 0 otherwise. Thus, $\mathbb{E}[\tilde{w}_x(\psi)] = w_x(\psi)$ and $\text{Var}(\tilde{w}_x(\psi)) = \mathcal{O}(K^{-1})$. Conditioned on these estimates, the step-1 target is $\tau_x(\tilde{w}) = \sum_{\psi \in \Psi} \tilde{w}_x(\psi) \mathcal{F}(\psi)$. By the conditional unbiasedness of the step-2 Hansen-Hurwitz estimator, $\mathbb{E}[\hat{\tau}_x \mid \tilde{w}] = \tau_x(\tilde{w})$. The total error decomposes orthogonally into step-2 noise ($A_{x,m}$) and step-1 structural error ($B_{x,K}$):

$$\Delta_x = \hat{\tau}_x - \tau_x = \underbrace{(\hat{\tau}_x - \tau_x(\tilde{w}))}_{A_{x,m}} + \underbrace{(\tau_x(\tilde{w}) - \tau_x)}_{B_{x,K}}. \quad (14)$$

Orthogonal Variance Decomposition. Conditioned on $\tilde{w}$, bounded weights ($|\tilde{w}| \leq W$) and $q_i \geq \varepsilon$ ensure $\mathbb{E}[A_{x,m}^2] = \mathbb{E}[\text{Var}(\hat{\tau}_x \mid \tilde{w})] \leq \mathbb{E}[\sum_i C_{i,x} / m_i] = \mathcal{O}(m^{-1})$. For step-1, since Ψ is finite, any linear combination yields $\mathbb{E}[B_{x,K}^2] = \text{Var}(B_{x,K}) = \mathcal{O}(K^{-1})$. By the law of iterated expectations, the cross-term vanishes: $\mathbb{E}[A_{x,m} B_{x,K}] = \mathbb{E}[B_{x,K} \mathbb{E}[A_{x,m} \mid \tilde{w}]] = 0$. Thus, the MSE decomposes additively: $\mathbb{E}[\Delta_x^2] = \mathcal{O}(m^{-1}) + \mathcal{O}(K^{-1})$. By Chebyshev's inequality, $\Delta_x = \mathcal{O}_p(m^{-1/2} + K^{-1/2})$, implying $\hat{\tau}_x \xrightarrow{p} \tau_x$. Since $\tau_{\text{count}} \geq c_0 > 0$, the Continuous Mapping Theorem yields $\hat{\tau}_{\text{avg}} \xrightarrow{p} \tau_{\text{avg}}$.

Asymptotic Bias. Define the high-probability event $\mathcal{E} = \{|\Delta_{\text{count}}| \leq c_0/2\}$ with indicator $\mathbb{I}_{\mathcal{E}}$. On $\mathcal{E}$, $\hat{\tau}_{\text{count}} \geq c_0/2 > 0$, yielding the exact algebraic identity:

$$(\hat{\tau}_{\text{avg}} - \tau_{\text{avg}})\mathbb{I}_{\mathcal{E}} = \left(\frac{\Delta_{\text{sum}}}{\tau_{\text{count}}} - \frac{\tau_{\text{sum}}\Delta_{\text{count}}}{\tau_{\text{count}}^2} + R \right) \mathbb{I}_{\mathcal{E}}, \quad (15)$$

where the remainder satisfies $|R|\mathbb{I}_{\mathcal{E}} \leq \mathcal{O}(1)|\Delta_{\text{sum}}\Delta_{\text{count}}| + \mathcal{O}(1)\Delta_{\text{count}}^2$. For the linear terms, unconditional unbiasedness ($\mathbb{E}[\Delta_x] = 0$) implies $\mathbb{E}[\Delta_x\mathbb{I}_{\mathcal{E}}] = -\mathbb{E}[\Delta_x\mathbb{I}_{\mathcal{E}^c}]$. Applying Chebyshev's inequality gives $\Pr(\mathcal{E}^c) \leq \frac{4}{c_0^2}\mathbb{E}[\Delta_{\text{count}}^2] = \mathcal{O}(m^{-1} + K^{-1})$. Applying Cauchy-Schwarz, $|\mathbb{E}[\Delta_x\mathbb{I}_{\mathcal{E}^c}]| \leq \sqrt{\mathbb{E}[\Delta_x^2]\Pr(\mathcal{E}^c)} = \mathcal{O}(m^{-1} + K^{-1})$. Similarly, applying Cauchy-Schwarz to R bounds its expectation by $\mathcal{O}(m^{-1} + K^{-1})$. Finally, on $\mathcal{E}^c$, the fallback ensures $\hat{\tau}_{\text{avg}} = 0$, bounding its absolute contribution by $|\tau_{\text{avg}}|\Pr(\mathcal{E}^c) = \mathcal{O}(m^{-1} + K^{-1})$. Summing all components yields $\mathbb{E}[\hat{\tau}_{\text{avg}}] - \tau_{\text{avg}} = \mathcal{O}(m^{-1}) + \mathcal{O}(K^{-1})$. Under $m = o(K)$, the $\mathcal{O}(K^{-1})$ term is strictly dominated, concluding $\mathbb{E}[\hat{\tau}_{\text{avg}}] - \tau_{\text{avg}} = \mathcal{O}(m^{-1})$. $\square$